YEUNGNAM UNIVERSITY · DEPARTMENT OF AUTOMOTIVE SOFTWARE · YEAR 3 FALL · MAJOR ELECTIVE

Automotive Sensor

Lecture 1 — What we are going to build:
a self-driving car, an embedded system, a development environment

Project-based course · two students enrolled · TOPST D3-G + PiRacer

Camera Ultrasonic Radar Lidar GNSS IMU

90-min lecture + 90-min lab

The next 90 minutes

A 90-minute lab follows the lecture. In the lab we build your development PC.

TIME	PART	TOPIC	KEY MESSAGE
00–10	A	**What we will build**	By the end of term, this car drives like this
10–35	B	**Self-driving cars**	Autonomy is a perceive–decide–act loop, and it starts at the sensor
35–60	C	**Embedded systems and vehicle computing**	A car is a computer network on wheels; our board is that network in miniature
60–80	D	**Development environment**	Write it on the PC, run it on the board
80–90	E	**Wrap-up · next week**	Next week we power up the board
+90	Lab	**Development PC setup**	Ubuntu 22.04 · GitHub lab-notebook repo · SSH key · role discussion

Three questions come up along the way. With two of you, both answer every time. You are not here to be right — you are here to give a reason.

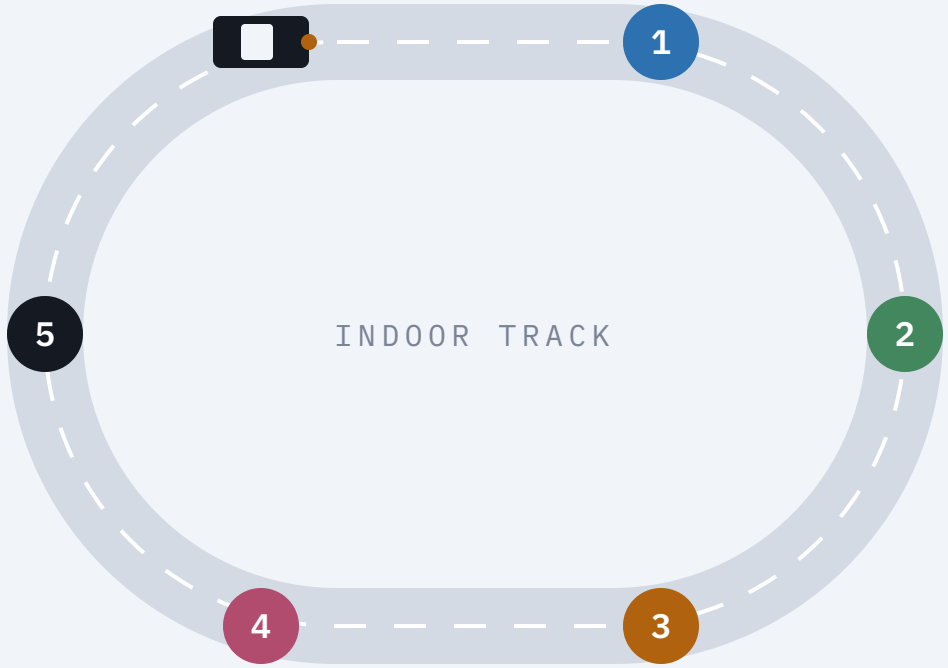
A

What we will build

> By the end of term, this car drives like this.

Week 15: this is what we demo on the indoor track

Five scenarios. Each one belongs to a different sensor.



01 Drive itself by following the lane
One full lap · MIPI-CSI camera → lane detection → steering

02 Stop for an obstacle ahead, then steer around it
Ultrasonic + 2D lidar · emergency braking (AEB) in miniature

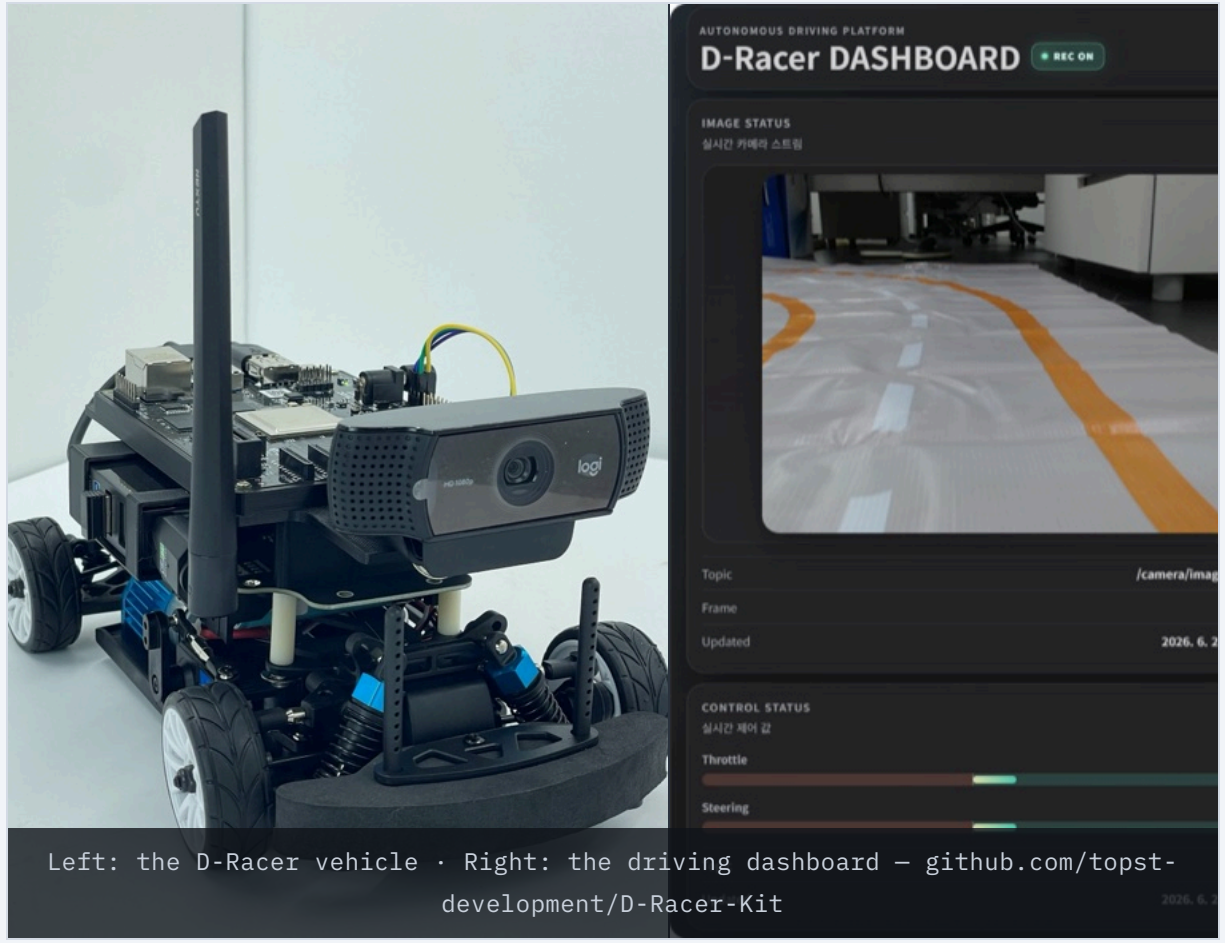
03 Measure how fast something approaches
HB100 Doppler radar module · relative velocity

04 Log the driving trajectory
GNSS + IMU · plot the path after the run

05 Show all six sensor streams live, together
Six ROS2 topics published at once + dashboard

The platform: D-Racer — TOPST D3-G on a PiRacer chassis

One 1/10-scale car, built jointly by the two of you.



Left: the D-Racer vehicle · Right: the driving dashboard — github.com/topst-development/D-Racer-Kit

Compute	TOPST D3-G — automotive SoC board (Telechips TCC8050)
Chassis	PiRacer — 1/10 scale, DC motor + steering servo
OS	Ubuntu 22.04
Middleware	ROS2 Humble — one node per sensor, joined by topics
Sensors	MIPI-CSI camera · ultrasonic · 2D lidar · radar · GNSS · IMU

> Official platform of the **2026 SEA:ME hackathon** (sponsored by Volkswagen Group Korea, hosted by Kookmin University). This term's work carries straight into the contest, or into capstone design.

Built on a Korean automotive SoC. Why not a Raspberry Pi: see Part C.

Two people, one car — shared platform, individual subsystems

Your assigned subsystem is **what you own in the project**, not what you learn. Theory and the mid-term cover all six sensors.

STUDENT A

Perception / Vision

MIPI-CSI camera, USB webcam — image pipeline, calibration, lane detection, steering angle

`/camera/image_raw · /lane/steering_angle`

STUDENT B

Ranging / Localization

Ultrasonic, Doppler radar, 2D lidar, GNSS, IMU — range and speed, obstacle detection, trajectory

`/ultrasonic/range · /scan · /radar/speed · /gps/fix · /imu/data`

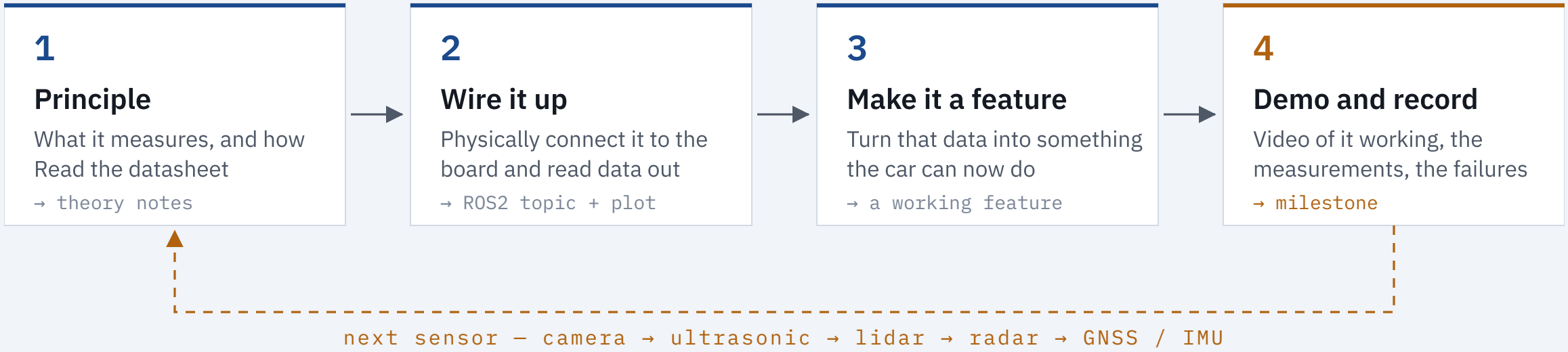
Shared by both: system architecture, the vehicle control node (motor and servo), integration, dashboard, final demo. Every lab meeting you explain your own subsystem to the other person, and in Lectures 8 and 14 you take an **oral check on your partner's sensors**.

PROJECT	WEEKLY LAB NOTEBOOK	MID-TERM CHECK	PARTICIPATION · PEER TEACHING
50%	**20%**	**20%**	**10%**

Milestones are not graded pass/fail. A failed attempt still scores full marks if the lab notebook holds the root-cause analysis and the next hypothesis. Half of every hardware project fails — that is normal. Do not hide it, record it.

Every new sensor goes through the same four steps

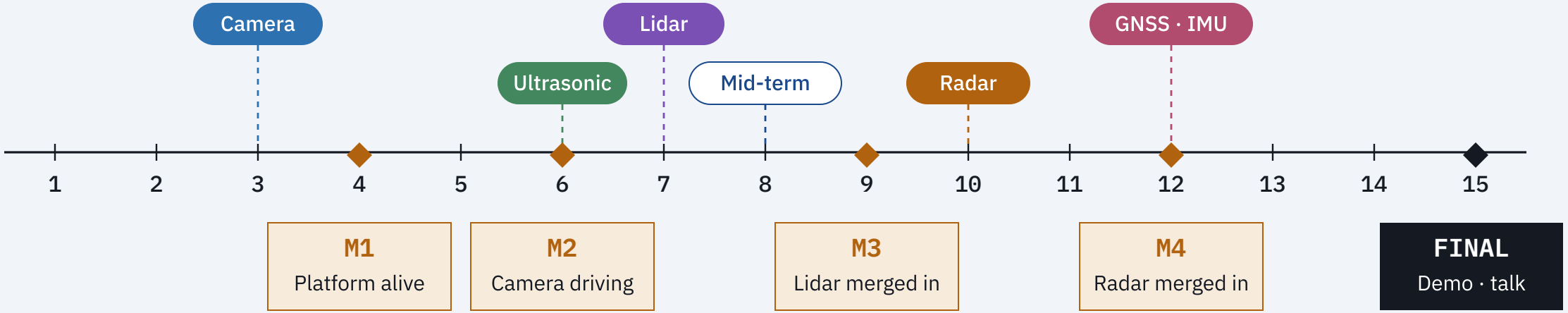
These four steps are the unit of learning in this course. We repeat them for camera, ultrasonic, lidar, radar and GNSS/IMU.



15 sessions — get the car rolling first, then add one sensor at a time

Each session is a 90-min lecture + 90-min lab. Most of the real development happens between sessions.

PHASE 0	PHASE 1	PHASE 2	PHASE 3	PHASE 4
Theory	**D3-G + camera** — boot · lane keeping	**+ lidar** — obstacle · avoid	**+ radar** — speed · warning	**Integrate** — final demo



10 – 35 MIN

B

Self-driving cars

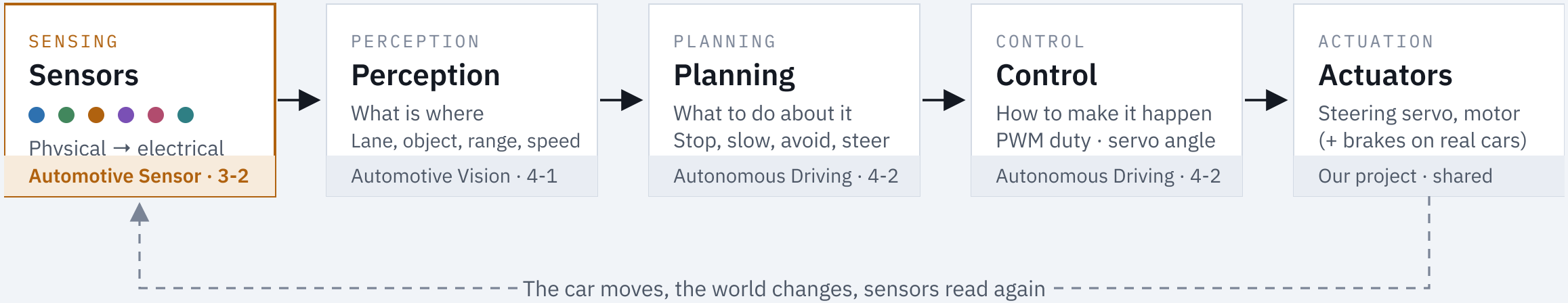
> Autonomy is a perceive–decide–act loop, and it starts at the sensor.

Autonomy is one loop — perceive, decide, act

This loop turns dozens of times a second. If any one box is late, the car reacts late.

The same order a person drives in — **look, decide, move your hands and feet.** Autonomy hands all three to a machine.

The grey strips name the course that owns each box. We own the leftmost one — everything up to the point where data reaches software.



A sensor never hands you "an obstacle" — it hands you a physical quantity

Every bit of interpretation is software's job. So knowing what is actually measured comes first.

SENSOR	PHYSICAL QUANTITY MEASURED	MEDIUM · WAVELENGTH	WHAT SOFTWARE RECEIVES
Camera	Intensity and colour of light	EM wave · 400–700 nm	2D pixel array (a frame)
Ultrasonic	Round-trip time of a sound pulse	Sound · about 40 kHz	One range value
Radar	Echo delay and frequency shift	EM wave · 24 / 77 GHz	Range · relative speed · angle
Lidar	Round-trip time of a laser pulse	EM wave · near-infrared	3D point cloud
GNSS	Time-of-arrival differences	EM wave · L band	Latitude · longitude · altitude
IMU	Acceleration and angular rate	Inertia (mechanics)	Attitude · relative motion

> No sensor tells you "there is a person ahead". It gives you **time, intensity, frequency** — nothing more. Turning those numbers into meaning is perception. Getting the numbers **right in the first place** is this course.

Automation levels 0–5 — the real border runs between L2 and L3

Per SAE J3016. What separates the levels is not capability but who watches the driving environment.

The driver watches the driving environment

The system watches the driving environment

L0
No automation

Warnings and brief help
Human drives, all of it

L1
Driver assistance

Steering or speed — one of them, not both
e.g. ACC, LKA

L2
Partial automation

Steering and speed both
Hands can come off, eyes cannot

L3
Conditional automation

Inside its conditions the system watches; hands back on request

L4
High automation

Inside a defined area (the ODD), no human input is needed

L5
Full automation

No limits on conditions. There need not be a steering wheel.

So where does our car sit? One indoor track, one lighting condition. The operating conditions are so narrow that a level number would mean nothing. What we build is a **scale reproduction of L1–L2 features (LKA, AEB)** — not a self-driving car.

The feature picks the sensor

Every ADAS feature on a production car leans on a different sensor. Our project builds a small version of each.

ADAS FEATURE	WHAT IT DOES	PRIMARY SENSOR	OUR COUNTERPART
ACC adaptive cruise	Holds a gap to the car ahead at a set speed	Forward radar (+ camera)	Closing speed from Doppler radar **L10–12**
AEB emergency braking	Brakes on its own when a crash is imminent	Radar + camera	Ultrasonic stop · lidar slow-down and avoid **L6 · 9**
LKA lane keeping	Nudges the steering to stay inside the lane	Forward camera	Lane detection → steering angle **L5–6**
BSD blind spot	Flags a car closing from the side or rear	Rear corner radar	Concept only **L11**
PAS parking aid	Reports range to close-in obstacles	Ultrasonic (a row in the bumper)	HC-SR04 detects ahead, then stops **L6**
Localization · routing	Where the car is right now	GNSS + IMU	Log the trajectory, then plot it **L12**

A pattern shows up — **need to know what it is, use a camera; how far and how fast, use radar; very close in, use ultrasonic.** Where one feature carries two sensors (AEB), the second one is there to cover the first one's failure.

Same problem, different answers — every company picks a different suite

If there were one right answer there would be one suite. Each of them traded away something different.

CASE A

Camera-first — Tesla



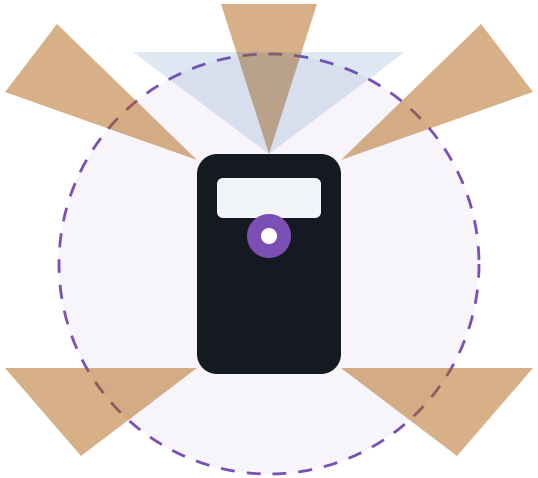
- Many cameras (front · side · rear)

Do with cameras what people do with eyes.

Cheap and information-rich, but weak in low light, bad weather, and range.

CASE B

Multi-sensor with lidar — Waymo



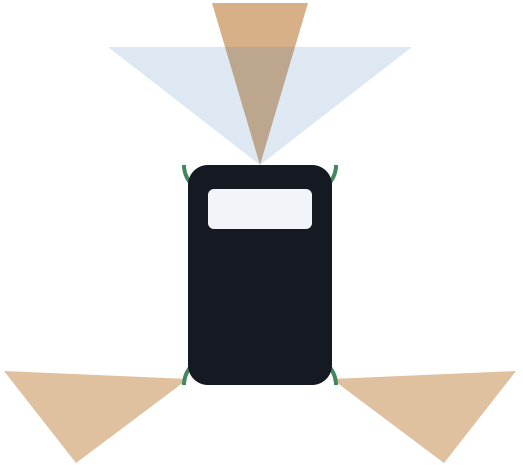
- Lidar
- Radar
- Camera

Stack sensors built on different physics.

One kind can fail and the others remain — at the cost of price, compute, alignment.

CASE C

Production ADAS — e.g. Hyundai



- Camera
- Radar
- Ultrasonic

Fit only the minimum each feature needs.

Forward camera + radar; ultrasonic close in. This is where cost and capability meet.

No single sensor wins everywhere

This table is the conclusion of the course and the starting point of sensor fusion.

	Night low light	Bad weather rain, fog, snow	Detect range maximum	Spatial detail shape sensing	Speed direct measure	Colour · text recognition	Relative cost more = pricier
Camera Light	●○○	●○○	●●○	●●●	●○○	●●●	●○○
Ultrasonic Sound	●●●	●●○	●○○	●○○	●○○	○○○	●○○
Radar Radio	●●●	●●●	●●●	●○○	●●●	○○○	●●○
Lidar Laser	●●●	●●○	●●●	●●●	●○○	○○○	●●●

More filled dots means stronger in that column — a three-step relative scale, not absolute numbers.
GNSS and IMU are absent: they never watch the environment. GNSS drops out under cover, and an IMU drifts with time.

QUESTION 01

Foggy night. Which sensor survives?

Both of you answer. The **reason** matters more than the pick — explain it by how that sensor's medium behaves in fog and in darkness.
Choose from camera, ultrasonic, radar, lidar, GNSS, IMU. Write your own answer into **lab notebook #1**.

Camera Ultrasonic Radar Lidar GNSS IMU

That is why we use several sensors

Overlapping sensors of different character beats making one sensor better.

CAMERA

What is it

Lane markings, signal colour, sign text, pedestrian or cyclist — it reads meaning.

Weak at: night, backlight, fog; range is guesswork

RADAR

How far and how fast

Range and relative speed in one shot, and almost untouched by rain, fog or darkness.

Weak at: shape. It has no idea what it hit

LIDAR

Where, to the centimetre

3D shape and range to centimetre accuracy — the reference for mapping and precise localization.

Weak at: bad weather. And it is expensive

Filling in each other's blanks is sensor fusion. And performance is not the only reason to overlap — when one sensor returns a **confidently wrong answer**, catching it takes a sensor built on different physics. That is redundancy.

We only survey fusion, in Lecture 13. The real fusion theory belongs to **Autonomous Driving (4-2)**. Our goal is the step before it — pulling in six kinds of data **accurately and at the same time**.

35 – 60 MIN

C

Embedded systems and vehicle computing

> A car is a computer network on wheels, and our board is that network in miniature.

An embedded system does one fixed job, by a fixed deadline

How is it different from your laptop? Not in performance — in what is assumed.

	GENERAL-PURPOSE PC	EMBEDDED SYSTEM
Purpose	Anything — the user decides	One fixed job — the designer decides
Time	Average throughput. Faster is better	**A deadline.** A late right answer is a wrong answer
Resources	Plentiful. Add more if you run short	Power, memory, heat and cost are all fixed
Environment	Indoors, room temperature, clean power	Freezing, baking, vibration, sagging supply
When it fails	You reboot	Someone can get hurt
Examples	Laptops, servers	Airbag ECU, ABS, camera module, **our D3-G**

Real-time does not mean fast. It means a guarantee that the work finishes by a stated time. A system that always finishes within 10 ms beats one that averages 1 ms but occasionally takes 200 ms. In a car what counts is not the average but the **worst case**.

Three kinds of computer — they differ in character, not in size

Second-year MCU Programming put the left column in your hands. We use the right one.

	MCU — ARDUINO UNO	MPU / SOC — RASPBERRY PI 4 B	AUTOMOTIVE SOC — TOPST D3-G
Processor	ATmega328P 8-bit AVR · 16 MHz · 1 core	Broadcom BCM2711 Cortex-A72 ×4 · 1.8 GHz	Telechips TCC8050 (Dolphin3) Cortex-A72 ×4 1.69 GHz · A53 ×4 1.45 GHz · **R5 600 MHz**
Memory	2 KB SRAM · 32 KB Flash	1–8 GB LPDDR4	4 GB or 8 GB LPDDR4X · 32 GB eMMC
Software	No OS (bare metal)	Linux	Linux (A72, A53) **+** FreeRTOS (R5)
Timing	**Deterministic** — predictable to the µs	**Non-deterministic** — the scheduler decides	**Both** — the cores are split between them
Good at	Toggling one pin at exactly the right moment	Video, networking, files; a deep ecosystem	Image processing and real-time control on one board

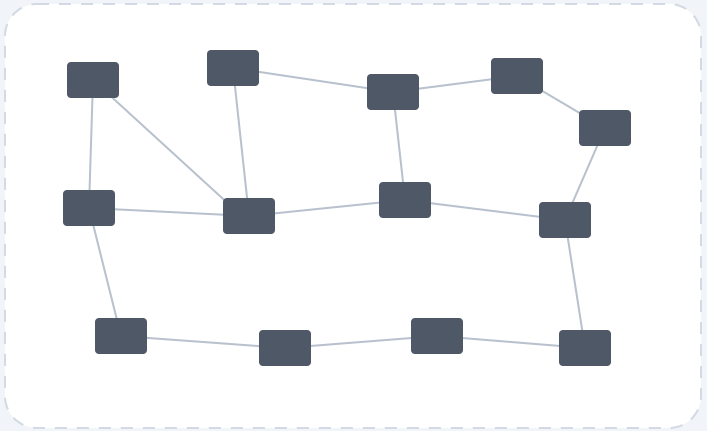
Sources: topst.ai/product/g/d3 · github.com/topst-development/Documentation (D3-G Specification) · raspberrypi.com product specs

The computer architecture inside a car is being redrawn right now

Bolting one computer onto every function ran into a wall of wiring and weight.

1980s – 2010s

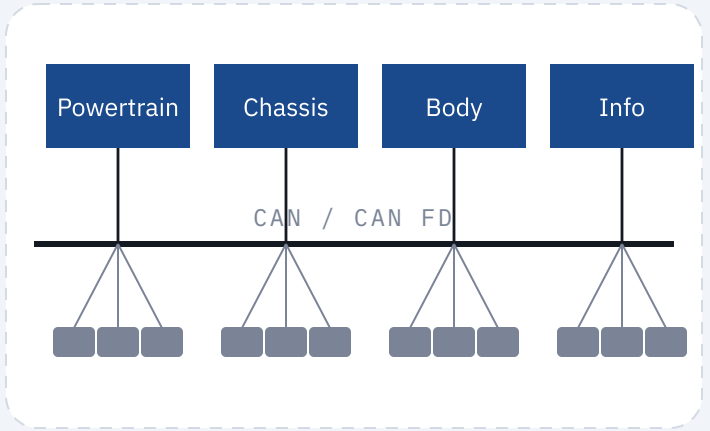
Distributed



One ECU per function. A luxury car passes 100. The problem is not compute but **wiring** — weight, cost, diagnosis and software updates all jam.

2015 –

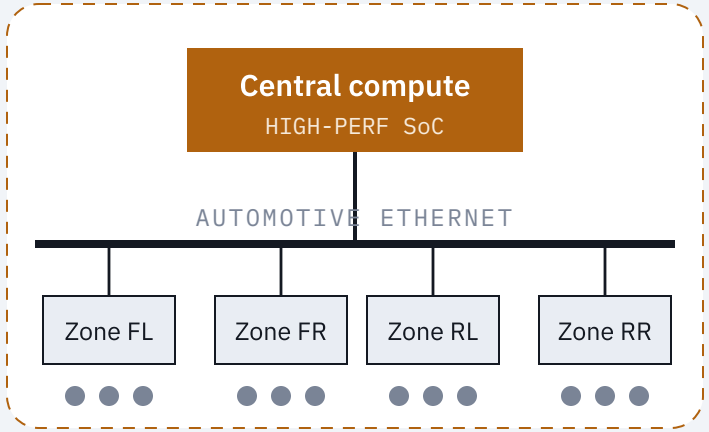
Domain



Group similar functions under one domain controller; the small ECUs sit beneath it. Less wiring — but more features cross domains.

2020 – ONGOING

Zonal · centralized

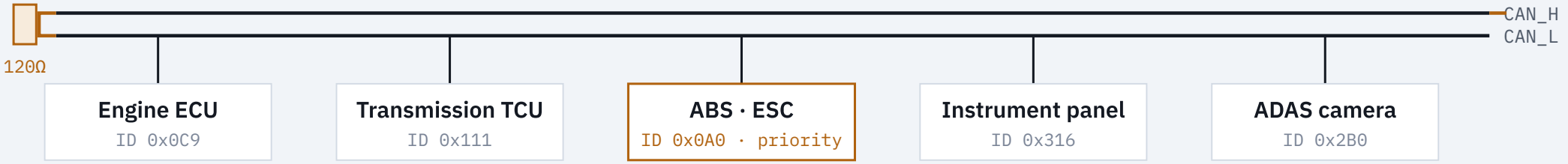


Wiring follows **location**; compute goes central. Sensors plug into the nearest zone and send only data over Ethernet — one car, one computer.

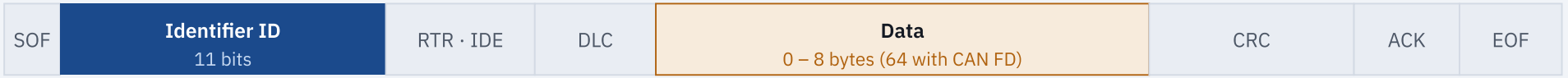
ECUs talk to each other over one pair of wires: CAN

There are no addresses. Everyone hears everyone, and each node keeps only what it needs.

Bits ride on the **voltage difference** between two wires — differential, so it survives engine-bay noise. Both ends terminate in 120 Ω.



CAN FRAME



The ID is the priority. Two nodes talking at once: the lower ID wins the bus, the loser backs off and resends. Brakes beat the radio.

> Classical CAN tops out at **1 Mbit/s** with **8 bytes** of payload. Camera video does not fit on that — which is why automotive Ethernet exists. **The D3-G provides 3 CAN channels** (transceivers live on a separate add-on board). We will not use them directly, but know CAN as the baseline of vehicle communication.

Our board — TOPST D3-G

A single-board computer carrying a Korean automotive SoC. 120 × 90 mm, powered from DC 5 V / 5 A.

COMPUTE

SoC	Telechips **TCC8050** (Dolphin3 family)
CPU	Cortex-A72 ×4 @ 1.69 GHz Cortex-A53 ×4 @ 1.45 GHz Cortex-R5 @ 600 MHz
GPU	PowerVR 9XTP GT9524 · 168 GFLOPS
Memory	LPDDR4X **4 GB / 8 GB** (32-bit ×2 channels)
Storage	eMMC 5.1 32 GB (64 GB option) + microSD
OS	Ubuntu 22.04 or Yocto (Kirkstone) · kernel 5.10

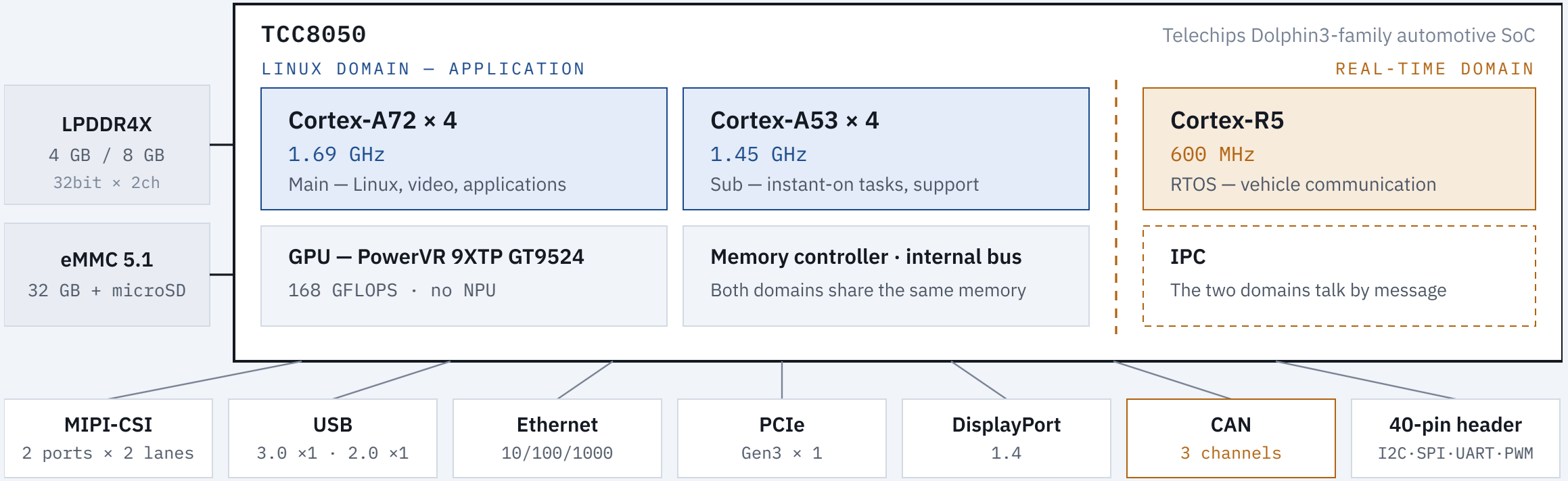
INTERFACES

Camera	MIPI-CSI, **2 ports × 2 lanes** (4-lane 20-pin option)
Header	**40-pin** (2×20, 2.54 mm), Raspberry Pi HAT compatible I2C · SPI · UART · I2S · PWM · GPIO
Vehicle bus	CAN, **3 channels** (transceivers on an add-on board)
Network	Ethernet 10/100/1000 Mbps ×1 · wireless by USB dongle
USB	3.0 Type-A ×1 · 2.0 Type-A ×1 · Type-C ×1 (firmware)
Other	DisplayPort 1.4 (HDMI via adapter) · PCIe Gen3 ×1

Sources: topst.ai/product/g/d3 · github.com/topst-development/Documentation → Single Board Computer / G Model / D3-G / Hardware / 1. Specification.md

One chip, cores of different character

Application cores running Linux and a real-time core that never touches Linux sit on the same die.



Why mix cores of different character on one chip

Linux is comfortable, but it **will not promise you a finish time.** And some jobs in a car need that promise.

APPLICATION CORES — CORTEX-A72 / A53

Linux runs here

- Filesystem, networking, drivers and libraries are all there already
- OpenCV, GStreamer and ROS2 run as-is
- But the scheduler **makes no promise** about when your code runs — sometimes it slips by tens of ms

`Good fit: image processing · path decisions · logging · comms`

REAL-TIME CORE — CORTEX-R5

An RTOS (FreeRTOS) runs here

- The OS is small and simple, so response time is predictable
- An interrupt is always serviced within a bounded time
- But none of Linux's comforts — files and packages are on you

`Good fit: vehicle comms · periodic control · safety monitoring`

This is not a choose-one problem. The old way was a Linux board and an MCU sitting side by side, joined by wires. An automotive SoC puts both on one die and joins them with internal messages (IPC). Part count, wiring and latency all drop together.

What makes this different from a Raspberry Pi

It is not a performance contest. It is whether the board arrives with what a car demands.

	RASPBERRY PI 4 B · 5	TOPST D3-G
SoC	BCM2711 — Cortex-A72 ×4 @ 1.8 GHz BCM2712 — Cortex-A76 ×4 @ 2.4 GHz (Pi 5)	TCC8050 — Cortex-A72 ×4 + A53 ×4 + R5
Real-time core	None — Linux and nothing else	**Cortex-R5 with an RTOS, built in**
CAN	Not on board — you add a HAT	**3 channels on board** (transceivers on an add-on)
Operating temp.	0 to +50 °C	−10 to +55 °C
Grade	Consumer / education / industrial. Not automotive-qualified (AEC-Q100)	An SoC family developed for automotive electronics
40-pin header	The original	Pin-compatible — HATs and example code carry over
Volume of material	**Overwhelming.** Search and you will usually find it	Mostly the official TOPST doc repo — if not, you ask

Get the distinction right. This is a development board carrying an automotive SoC, not a part you can bolt to a car. Chip grade and board grade differ — this board itself runs from −10 to +55 °C. A real vehicle is another problem.

Why learn on an automotive SoC

There is an easier road. Four reasons we take this one instead.

- **You learn the structure.** Heterogeneous cores, CAN, the boot chain — none of it exists on a Raspberry Pi. Most of what makes automotive software hard comes out of that structure.
- **The industry is heading this way.** The "central compute" box in a zonal architecture is filled by this class of chip. Think back to the third panel on the previous slide.
- **You will actually meet this chip here.** Telechips is a Korean automotive semiconductor company and TOPST is its official development platform. Work in automotive software in Korea and you will handle this family.
- **It carries past the semester.** The same platform runs into the SEA:ME hackathon, capstone design, Automotive Vision (4-1) and Autonomous Driving (4-2). Your GitHub repo stays with you.

> One thing to brace for — **you will hit problems that a search engine cannot answer.** On a Raspberry Pi, someone has already been there. On this board, maybe not. Reading the official docs, reading the logs, and asking — that is what the job actually is.

QUESTION 02

Camera processing and motor control — which core for each?

Pick between the Cortex-A72 (Linux) and the Cortex-R5 (RTOS), and say **why**.

Then one more: if you put both on the A72, what breaks?

CORTEX-A72 · A53

Linux · rich libraries · no guaranteed response time

CORTEX-R5

RTOS · predictable response time · no comforts

60 – 80 MIN

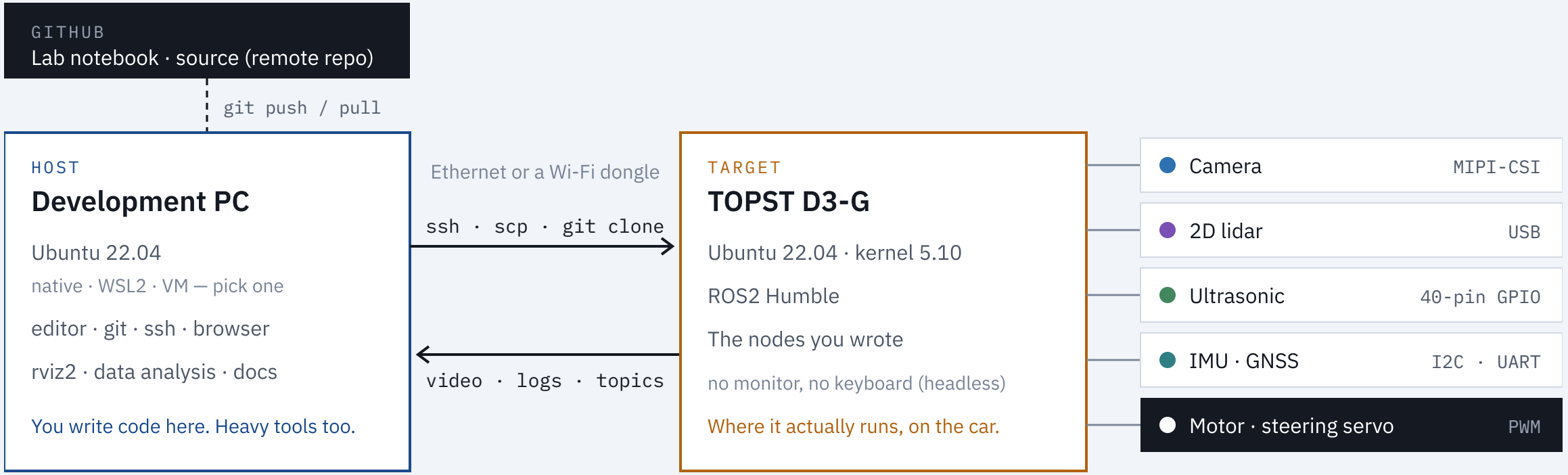
D

Development environment

> Write it on the PC, run it on the board.
> Linux, SSH, ROS2 and Git are what joins the two.

You work on two computers — host and target

No monitor and no keyboard go on the board. You go in over the network.



What you will do all term — **write on the left, run on the right, read the results back on the left.**

Why Linux

Not a matter of taste. Without these four, the car does not roll inside one semester.

- **The drivers already exist.** Camera over V4L2, lidar over USB serial, sensors over I2C and SPI — the kernel already exposes all of it through standard interfaces. We just open `/dev/video0`.
- **The tools and packages exist.** One line of `apt` brings in Python, OpenCV and GStreamer. Building that yourself would eat the whole semester.
- **ROS2 lives on Linux.** The reference environment for ROS2 Humble is Ubuntu 22.04. That is the whole reason we use 22.04.
- **You can get in remotely.** SSH in, read the logs, move files back and forth. You fix a board that is on the car from your desk.

> The price is just as clear — **Linux will not promise you a finish time.** Exactly as in Part C. That is why an automotive SoC parks a real-time core next to Linux. So far it is the only way to get convenience and guarantees at once.

The screen you will stare at longest this term is the **terminal**. `cd` `ls` `apt` `ssh` `vim`, file permissions — that much should be in your fingers.

What happens when you power up the board

Here is where it parts company with Linux on a PC: every stage of the boot is something we put there.

01
Power · boot ROM
Fixed code burned into the chip. It decides where to boot from.

→

02
Bootloader
Wakes up memory and loads the kernel.
`U-Boot`

→

03
Linux kernel
Brings up drivers. What hardware sits where is described by `the device tree`

→

04
rootfs · init
The filesystem mounts and services start.
`Ubuntu 22.04`

→

05
Our code
The ROS2 nodes come up. The car starts to move.

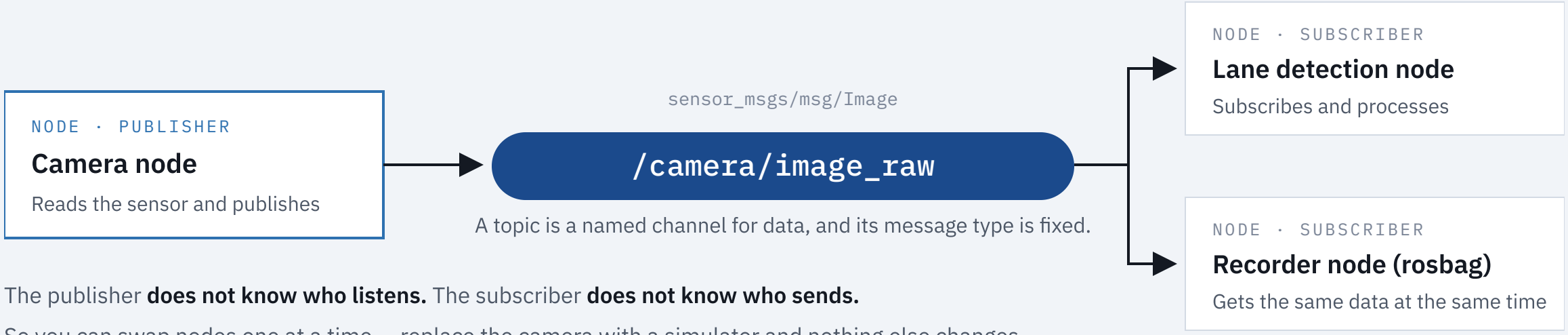
Flashing a microSD image puts all three on

On a PC the manufacturer did all of this already. On an embedded board **we lay it down ourselves** — so there are five places it can fail.

Read the boot log and you can see where it stopped — **in Lecture 2 we flash it, boot it, and SSH in.**

ROS2 on one slide — nodes and topics

You do not write one program. Small programs — nodes — talk over named channels called topics.



The publisher **does not know who listens.** The subscriber **does not know who sends.**

So you can swap nodes one at a time — replace the camera with a simulator and nothing else changes.

Why not write it all as one program

Six sensors, two people. Split nodes is what falls out of those two facts.

ISOLATION

One dies, the rest live

Each node is its own process. The lidar driver can crash and the camera keeps driving the car. And **you can see immediately whose code broke.**

DIVISION OF WORK

Two people never touch the same file

A publishes `/camera/image_raw`, B publishes `/scan`. All you agreed on was **the topic name and the message type**.

TOOLING

See it, record it, replay it

`rviz2` shows the lidar point cloud with your own eyes; `rosbag` records a whole run so you can replay it at your desk.

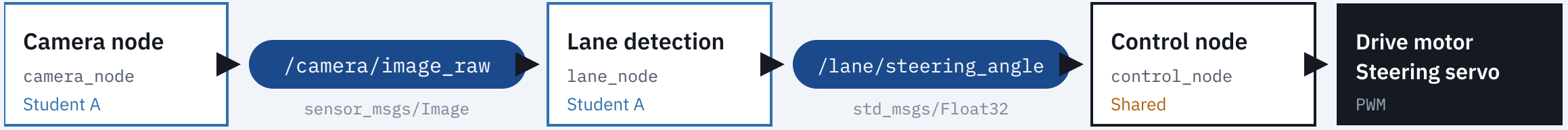
rosbag matters more than usual in this course. Sensor problems mostly appear only in that one moment. Record the run and you can replay the failure as many times as it takes. **A failure you did not record cannot be analysed** — and this is also where lab notebook evidence comes from.

The goal here is not to make you a ROS2 expert. If **node, topic, publish and subscribe** are enough to get sensor data flowing, that is enough.

The system we build — the node and topic graph

The solid path is finished by Lecture 6. The dashed boxes join it one at a time after that.

BUILDING NOW — LECTURES 3 TO 6



ADDED LATER — DASHED



Commit every week — that repository is the deliverable

The lab notebook is 20% of the grade, but it is the one thing that outlives the semester.

REPOSITORY LAYOUT (EXAMPLE)

```
lab-notebook/
├── README.md
├── notes/
│   ├── week01.md
│   ├── week02.md
│   └── ...
├── src/      ROS2 packages
└── data/     measurements · logs · photos
```

Keep **the record and the code in one repo**. Split them and neither one gets maintained.

WHAT ONE LAB-NOTEBOOK ENTRY CONTAINS

1 Theory	Three to five lines on this week. **Name what you did not understand**
2 Progress	Done / stuck on / what you tried / plan for next week
3 Evidence	Measurements, plots, screenshots, scope captures, **failure logs**
4 Source code	Committed to the same repository

Graded on **specificity**, not on length. Not "it didn't work" but "I fed it 3.3 V, the echo pin stayed dead, so I tried 5 V".

> In a job interview, a graduate application or a hackathon, what proves "I built this" is not a transcript — it is **this repository**. And commit dates cannot be manufactured afterwards: fifteen weeks of weekly commits are their own evidence.

80 – 90 MIN

E

Wrap-up · next week

> Next week we power up the board.

Today on one slide

Five sentences are all you need to keep.

PART	TOPIC	WHAT TO KEEP
A	**What we will build**	Get the car rolling first, then add sensors one at a time. Every sensor repeats **principle → wire up → feature → record**.
B	**Self-driving cars**	Autonomy is a **perceive–decide–act loop**. Sensors give physical quantities only. No sensor wins everywhere, so we overlap several.
C	**Embedded · vehicle computing**	Embedded is defined by the **deadline**, not by performance. An automotive SoC puts a Linux core and a real-time core on one chip.
D	**Development environment**	**Write on the PC, run on the board.** Nodes talk over topics, so two people build separately and still fit together.
E	**Attitude**	Half of any hardware project fails. **The person who records the failures gets the best grade here.**

Whatever you did **not** understand today, put it into lab notebook #1 as it stands — here that is data, not a deduction.

Lecture 2 — we power up the board

The session where theory turns into hardware. You use the development PC you set up today.

LECTURE · 90 MIN

- Embedded Linux structure — bootloader, kernel, rootfs, **device tree**
- Host–target workflow (cross development, flashing an image, serial console)
- ROS2 architecture — nodes, topics, messages, **launch files**
- Sensor interfaces at a glance — GPIO, I2C, UART, SPI, MIPI-CSI

LAB · 90 MIN

- Flash the D3-G boot image (microSD)
- First boot · read the boot log together
- Configure the network → **SSH in**
- Verify the ROS2 install → two nodes, `talker / listener`

Due in Lecture 2 — ① roles settled (A vision / B ranging and localization) ② draft system architecture: the list of **node names, topic names and message types** the two of you agreed on

Bring — your laptop (with today's Ubuntu setup) · a microSD card reader · an Ethernet cable if you have one. The board and the microSD come from the department.

Lab notebook #1 — before next week's class

Three things, written into `notes/week01.md` and committed. It does not need to be long.

- **What you set up today.** Which route you took for Ubuntu (native / WSL2 / VM) and **why you chose it**. What blocked you during install and how you got past it. One screenshot is enough.
- **Your own answer to "foggy night, which sensor survives?"** Pick one sensor and **argue it from the physics**. If your view changed during class, say what changed it.
- **One paragraph on which role you want.** A (perception / vision) or B (ranging / localization), and why that side. If you want both, write that. We settle it in Lecture 2.

HOW TO SUBMIT

Commit to your own GitHub repo, then **send me the repository URL once**. After that, committing is all you do.

HOW IT IS GRADED

On **specificity**, not length. A notebook that records where you got stuck scores higher than a tidy one that doesn't.

LAB / 90 MINUTES

Lab, 90 minutes — build the development PC

Finish it today and we can power up the board next week. If you get stuck, we unblock it on the spot.

TIME	TASK	DONE MEANS
00–10	**Check what you have**	Your laptop's OS, free disk space and admin rights checked. Decide which of the three routes you will take
10–45	**Install Ubuntu 22.04**	A terminal opens and `lsb_release -a` prints 22.04
45–70	**GitHub · SSH key**	The lab-notebook repo exists and `ssh -T git@github.com` authenticates
70–85	**Role discussion**	You tell each other which side you want — A (perception / vision) or B (ranging / localization) — and sort out any clash
85–90	**Close out**	Lab notebook #1 items confirmed · what to bring to Lecture 2 confirmed

> It is fine if the install does not finish today. **What is not fine is not knowing where it stopped.** Copy the error message verbatim into your lab notebook and we pick it up next week. Start the habit of capturing the screen now.

Ubuntu 22.04 — pick one of three routes

22.04 is the reference environment for ROS2 Humble. Do not bump the version on your own.

ROUTE	UPSIDE	DOWNSIDE	PICK IT IF
Native install dual boot	Full speed, full device access	Needs partitioning; hard to undo	you plan to keep using Linux
WSL2 Windows 11 / 10	Easiest install, Windows stays	USB needs `usbipd-win`. GUI tools limited	**the safest starting point**
Virtual machine VirtualBox etc.	Safe; snapshots roll back	Slow, and hungry for memory	your laptop has spec to spare

CREATE AND REGISTER AN SSH KEY

```
ssh-keygen -t ed25519 -C "studentid@yu.ac.kr"
cat ~/.ssh/id_ed25519.pub
# paste the output into GitHub →
#   Settings → SSH and GPG keys → New SSH key
ssh -T git@github.com   # check it works
```

CREATE THE LAB-NOTEBOOK REPO

```
# create lab-notebook on GitHub (Public)
git clone git@github.com:<account>/lab-notebook.git
cd lab-notebook && mkdir -p notes src data
git add . && git commit -m "init"
git push
```

Make it Public — a portfolio has to be readable. Never commit passwords or personal data.